

scenario where a single model is capable of holding other models. You need to use `hasMany()` method, which accepts several argument such as local and foreign key.

To work with One To Many (Inverse), you should define a relationship function on the child model, which will ultimately call the `belongsTo()` method. Talking about Many To Many relationships, you need to perform a complicated process by creating three database tables such as users, roles, and role_user. The `role_user` table inherits from the model names and it consists of `user_id` and `role_id` columns. the relationship returns the result of the `belongsToMany()` method. The user roles can be accessed by employing the roles dynamic property. The Has Many Through provides a shortcut to access various relations by using `hasManyThrough()` method, which accepts three arguments. The official documentation provides a detailed overview of polymorphic relations, querying relations, eager loading including inserting and updating of related models.

Collections

Eloquent make use of an instance of `Illuminate\Database\Eloquent\Collection` object to return multiple result sets. The Eloquent collection object extends the Laravel base collection, which helps you to use a wide range of methods. Actually, collections also serve the purpose of iterators. It provides an ability for you to loop over them as you work with simple PHP arrays. As a developer, you should note that collections are more powerful than arrays and it exposes several map/reduce operations. While, most of the collection methods return a new instance, the methods such as `flatten`, `pluck`, `zip`, `flip`, and `keys`, collapse return an instance of base collection. Some of the methods found in the base collection are `all()`, `average()`, `avg()`, `chunk()`, `collapse()`, `combine()`, `concat()`, `contains()`, `containsStrict()`, `count()`, `crossJoin()`, `dd()`, `diff()`, `diffKeys()`, `intersect()`, `isEmpty()`, `isNotEmpty()`, `max()`, `median()`, `merge()`, `min()`, `toArray()`, `toJson()`, `transform()`, `union()`, and `unique()`. You will find information about all the methods on the official Laravel documentation page.

Accessors & Mutators

Accessors and Mutators are mainly used to format Eloquent attribute values when you create them on model instances. The framework can also cast text fields to JSON and cast date fields to Carbon instances depending